Complete AI Viva Question Bank with Answers

1. What is Artificial Intelligence?

Artificial Intelligence (AI) is the simulation of human intelligence by machines so they can think, learn, reason, solve problems, and make decisions like humans.

---

2. What are the goals of AI?

- Learning
- Reasoning
- Problem solving
- Decision making
- Perception
- Natural Language Processing
- Planning
- Robotics

---

3. What are the foundations of AI?

- Mathematics
- Computer Science
- Psychology
- Philosophy
- Economics
- Neuroscience
- Linguistics
- Control Theory

---

4. What is the history of AI?

- 1950: Alan Turing introduced Turing Test
- 1956: John McCarthy coined the term AI

- AI Winter: progress slowed due to lack of funding
- Modern AI uses Machine Learning and Deep Learning

---

5. Who introduced the Turing Test?

Alan Turing introduced the Turing Test in 1950.

---

6. What is the Turing Test?

It is a test used to check whether a machine can behave like a human in conversation.

If a human cannot distinguish machine from human, the machine passes the test.

---

7. Who coined the term Artificial Intelligence?

John McCarthy coined the term Artificial Intelligence in 1956.

---

8. What are the benefits of AI?

- Automation
- High accuracy
- Faster work
- 24/7 availability
- Reduced human effort
- Better decision making

---

9. What are the risks of AI?

- Job loss
- Privacy issues
- Bias in decision making
- Security threats
- Ethical concerns
- High dependency on machines

---

10. What is an Intelligent Agent?

An intelligent agent is an entity that observes the environment using sensors and acts using actuators.

Example: Robot, self-driving car

---

11. What is a Rational Agent?

A rational agent always chooses the best action to maximize performance and achieve the best result.

---

12. Difference between Agent and Rational Agent?

Agent → simply performs actions

Rational Agent → performs the best possible action

---

13. What is PEAS representation?

PEAS is used to describe an intelligent agent.

It stands for:

- Performance Measure
- Environment
- Actuators
- Sensors

---

14. Expand PEAS

P → Performance Measure
E → Environment
A → Actuators
S → Sensors

---

15. What are sensors and actuators?

Sensors collect information from environment.

Actuators perform actions in environment.

Example:

Robot:

Camera = Sensor

Wheels = Actuator

---

16. What are the types of agents?

- Simple Reflex Agent
- Model-Based Agent
- Goal-Based Agent
- Utility-Based Agent
- Learning Agent

---

17. What is a Simple Reflex Agent?

It works only on current condition using condition-action rules.

Example:

If obstacle → turn left

---

18. What is a Model-Based Agent?

It keeps track of past information and current state before making decisions.

---

19. What is a Goal-Based Agent?

It takes actions to achieve a specific goal.

---

20. What is a Utility-Based Agent?

It chooses the action that gives maximum usefulness or benefit.

---

21. What is a Learning Agent?

It improves performance using past experience and learning.

---

22. What are the types of environments?

- Fully observable / Partially observable
- Deterministic / Stochastic
- Static / Dynamic
- Discrete / Continuous
- Single-agent / Multi-agent

---

23. Difference between fully observable and partially observable environment?

Fully observable → complete information available

Partially observable → some information hidden

Example: Chess vs Poker

---

24. Difference between deterministic and stochastic environment?

Deterministic → same action gives same result

Stochastic → result may vary due to randomness

---

25. Difference between static and dynamic environment?

Static → environment does not change while agent thinks

Dynamic → environment changes continuously

---

26. Difference between discrete and continuous environment?

Discrete → fixed states/actions

Continuous → infinite possible values

---

27. Difference between single-agent and multi-agent environment?

Single-agent → only one agent involved

Multi-agent → multiple agents interact

---

28. What is problem solving in AI?

Finding a sequence of actions from initial state to goal state.

---

29. What is a Problem-Solving Agent?

An agent that searches for a solution using search algorithms.

---

30. What are the components of problem formulation?

- Initial State
- Actions
- Transition Model
- Goal Test
- Path Cost

---

31. What is an initial state?

The starting point of the problem.

---

32. What is goal state?

The final desired state to be reached.

---

33. What is path cost?

The total cost required to reach goal from start.

---

34. What is state space?

The set of all possible states in a problem.

---

35. What is a search tree?

A tree formed while exploring possible states.

---

36. Difference between state space and search tree?

State Space → all possible states

Search Tree → explored paths only

---

37. What is a search algorithm?

An algorithm used to move from initial state to goal state.

---

38. What is uninformed search?

Search without heuristic knowledge.

Example:

- BFS
- DFS

---

39. Examples of uninformed search?

- Breadth First Search
- Depth First Search
- Uniform Cost Search

---

40. What is informed search?

Search using heuristic knowledge.

Example:

- A*
- Greedy Best First Search

---

41. Examples of informed search?

- A*
- Greedy Best First Search
- Hill Climbing

---

42. What is heuristic function?

A function that estimates how close the current state is to goal state.

---

43. What is local search?

Search that uses only current state, not the full path.

Example: Hill Climbing

---

44. What is optimization problem?

Finding the best possible solution among many solutions.

---

45. What is Hill Climbing?

A local search algorithm that moves toward better neighboring states.

---

46. Difference between informed and uninformed search?

Informed → uses heuristic

Uninformed → does not use heuristic

---

47. Difference between BFS and DFS?

DFS → goes depth-wise first

BFS → goes level-wise first

DFS uses recursion/stack

BFS uses queue

---

48. Why does BFS use queue?

Because queue follows FIFO (First In First Out), which helps level-order traversal.

---

49. Why does DFS use recursion?

Because DFS goes deep first and recursion naturally handles backtracking.

---

50. Can DFS be implemented using stack?

Yes.

DFS can be implemented using:

- Recursion
- Stack

But recursive DFS is preferred when aim asks recursive algorithm.

---

51. What is adjacency matrix?

Adjacency matrix is a 2D array used to represent a graph.

If there is a connection between node i and node j, we write 1, otherwise 0.

Example:

0 1 1 0

1 0 0 1

1 0 0 1

0 1 1 0

---

52. How do you identify an undirected graph?

In an undirected graph, adjacency matrix is symmetric.

matrix[i][j] = matrix[j][i]

If both are equal, graph is undirected.

---

53. What is a visited array?

Visited array keeps track of already visited nodes.

It prevents revisiting the same node again.

Example:

visited = [False, False, False, False]

---

54. Time complexity of BFS?

Using adjacency matrix:

$O(V^2)$

Where V = number of vertices

---

55. Time complexity of DFS?

Using adjacency matrix:

$O(V^2)$

---

56. Can BFS find shortest path?

Yes.

BFS can find shortest path in an unweighted graph.

---

57. Can DFS find shortest path?

No.

DFS does not guarantee shortest path.

---

58. What is graph traversal?

Graph traversal means visiting all vertices (nodes) of a graph systematically.

Main methods:

- DFS
- BFS

---

59. Explain DFS

DFS means Depth First Search.

It goes deep into one branch first before moving to another branch.

It uses recursion or stack.

---

60. Explain BFS

BFS means Breadth First Search.

It visits all neighboring nodes first, then moves deeper.

It uses queue.

---

61. Difference between DFS and BFS?

DFS:

- Depth-wise
- Uses recursion/stack
- Less memory
- No shortest path guarantee

BFS:

- Level-wise
- Uses queue
- More memory
- Gives shortest path

---

62. Why is DFS recursive?

Because DFS explores deeply first and recursion handles deep traversal and backtracking naturally.

---

63. Where is recursion in DFS code?

Here:

dfs(matrix, node, visited, dfs_list)

Function calls itself.

This is recursion.

Very important viva question.

---

64. Why reset visited before BFS?

Because DFS already marks all nodes visited.

If we do not reset it, BFS will think nodes are already visited and traversal will fail.

---

65. What is adjacency matrix representation?

It is representing graph using rows and columns.

Each row tells which nodes are connected to that vertex.

---

66. What is symmetric matrix in graph?

If:

matrix[i][j] = matrix[j][i]

then matrix is symmetric.

This represents an undirected graph.

---

67. Why is your graph undirected?

Because adjacency matrix is symmetric.

If node 1 is connected to node 2, then node 2 is also connected to node 1.

---

68. Can DFS be iterative?

Yes.

DFS can be done using stack without recursion.

But recursive DFS is preferred here.

---

69. What is stack-based DFS?

DFS using stack instead of recursion.

Push node → Pop node → Visit → Repeat

It follows LIFO.

---

70. Which DFS is preferred in this practical and why?

Recursive DFS.

Because the aim specifically says:

“develop a recursive algorithm”

So recursion is expected.

---

71. What is queue in BFS?

Queue stores nodes to be visited.

It follows FIFO:

First In First Out

---

72. Explain FIFO

FIFO means:

First In First Out

Example:

Queue of people

First person enters first → leaves first

Used in BFS

---

73. Explain LIFO

LIFO means:

Last In First Out

Example:

Stack of plates

Last plate placed → removed first

Used in DFS stack version

---

74. What is A* Algorithm?

A* is a best-first search algorithm used to find shortest path from start state to goal state.

It uses:

f(n)=g(n)+h(n)

---

75. Where is A* used?

- 8 Puzzle Problem
- GPS navigation
- Maze solving
- Robot movement
- Path finding in games

---

76. What is the 8 Puzzle Problem?

It is a puzzle with:

- 8 numbered tiles
- 1 blank space (_)

Goal is to move tiles and reach the target arrangement.

---

77. What is the blank space (_) used for?

Blank space is the movable empty space used to shift tiles.

Only blank space can move.

---

78. Explain the formula f(n) = g(n) + h(n)

f(n)=g(n)+h(n)

g(n) = cost to reach current state

h(n) = estimated cost to reach goal

f(n) = total estimated cost

A* chooses minimum f(n)

---

79. What is g(n)?

Cost from start node to current node.

In your code:

level

means number of moves taken.

---

80. What is h(n)?

Heuristic value.

Estimated distance from current state to goal state.

---

81. What is f(n)?

Total estimated cost.

Smaller f(n) means better move.

---

82. What heuristic is used in your code?

Number of misplaced tiles.

This is the heuristic function used.

---

83. What is misplaced tile heuristic?

It counts how many tiles are in the wrong position compared to goal state.

Blank space is ignored.

---

84. What is OPEN list?

OPEN list stores states that are yet to be explored.

---

85. What is CLOSED list?

CLOSED list stores states that are already explored.

---

86. Why is OPEN list sorted?

To choose the state with minimum f(n).

A* always selects best possible next move.

---

87. Why do we choose minimum f(n)?

Because smaller f(n) means state is more promising and closer to goal.

---

88. What are child states?

Possible next states generated by moving blank tile.

Example:

Move blank:

- left
- right
- up
- down

---

89. What is goal state in A*?

Final desired arrangement of puzzle.

Example:

1 2 3

4 5 6

7 8 _

---

90. What is deepcopy?

deepcopy() creates a completely separate duplicate copy of an object.

Used from:

import copy

---

91. Why is deepcopy used?

To create new child states without changing original puzzle.

Very important question.

---

92. What happens if deepcopy is not used?

Original puzzle gets modified.

This breaks the algorithm and gives wrong results.

---

93. Can A* guarantee optimal solution?

Yes, if heuristic is admissible.

---

94. What is admissible heuristic?

A heuristic that never overestimates the actual cost.

It always gives safe estimate.

---

95. Difference between A* and Greedy Best First Search?

A* uses:

f(n)=g(n)+h(n)

Greedy uses only:

h(n)

A* is more accurate and optimal.

---

96. What is Greedy Algorithm?

Greedy Algorithm chooses the best immediate (local) option at every step, hoping to get the overall best (global) solution.

It does not reconsider previous choices.

---

97. Why is it called greedy?

Because it always takes the best-looking choice immediately without thinking about future consequences.

Like “best now” approach.

---

98. Difference between Greedy and Dynamic Programming?

Greedy:

- Takes immediate best choice
- Does not store previous results
- Faster but may fail sometimes

Dynamic Programming:

- Solves all possibilities
- Stores previous results
- More accurate for complex problems

Very important viva question.

---

99. What are applications of Greedy Algorithm?

- Dijkstra Algorithm
- Prim’s Algorithm
- Kruskal Algorithm
- Job Scheduling
- Huffman Coding
- Selection Sort

Dijkstra Algorithm

---

100. What is Dijkstra Algorithm?

It finds the shortest path from one source node to all other nodes in a weighted graph.

---

101. What is Single Source Shortest Path Problem?

Finding shortest path from one starting node to every other node.

This is what Dijkstra solves.

---

102. Can Dijkstra handle negative weights?

No.

Very common trap question.

---

103. Why can Dijkstra not handle negative weights?

Because greedy choice becomes incorrect when negative edges exist.

It may produce wrong shortest path.

---

104. Time complexity of Dijkstra?

Simple implementation:

$O(V^2)$

where V = vertices

---

105. Difference between Dijkstra and BFS?

BFS:

- For unweighted graph

Dijkstra:

- For weighted graph

Dijkstra handles weights, BFS does not.

Prim's Algorithm

---

106. What is Prim's Algorithm?

Prim's Algorithm finds Minimum Spanning Tree (MST) by growing one tree step by step.

---

107. What is Minimum Spanning Tree (MST)?

It is a tree that:

- connects all vertices
- has minimum total edge weight
- contains no cycles

Very important question.

---

108. Properties of MST?

- All vertices connected
- No cycle
- Minimum total weight
- Exactly V−1 edges

---

109. Difference between Prim and Dijkstra?

Prim:

- Finds MST

Dijkstra:

- Finds shortest path

Very favorite viva question.

---

110. Time complexity of Prim's?

Simple implementation:

$O(V^2)$

Kruskal Algorithm

---

111. What is Kruskal Algorithm?

It finds MST by selecting the smallest edge first without forming a cycle.

It is edge-based.

---

112. Difference between Prim and Kruskal?

Prim:

- Vertex-based
- Grows one tree

Kruskal:

- Edge-based
- Selects smallest edges first

Very common viva question.

---

113. How is cycle detected in Kruskal?

Using Disjoint Set / Union-Find.

---

114. What is Union-Find?

A data structure used to detect cycles and manage connected components.

It uses:

- Union
- Find

---

115. Time complexity of Kruskal?

Usually:

O(E log E)

where E = edges

Unit 3 – Adversarial Search and Games

---

116. What is Game Theory?

Game Theory is the study of decision-making in competitive situations where one player's gain may affect another player.

---

117. What is adversarial search?

Search used in competitive games where one player tries to defeat another.

Example:

Chess, Tic-Tac-Toe

---

118. Examples of adversarial search?

- Chess
- Tic-Tac-Toe
- Checkers
- Connect Four

---

119. What is Minimax Algorithm?

Minimax is used in two-player games where one player tries to maximize winning and the other tries to minimize it.

---

120. Why is Minimax used?

To make the best possible decision against an intelligent opponent.

---

121. What is Alpha-Beta Pruning?

It is an optimization of Minimax that removes unnecessary branches from search tree.

---

122. Why is Alpha-Beta pruning used?

To reduce search time and improve efficiency.

Same result, less work.

---

123. Difference between Minimax and Alpha-Beta pruning?

Minimax:

- explores all branches

Alpha-Beta:

- skips unnecessary branches

Alpha-Beta is faster.

---

124. What is Monte Carlo Tree Search?

It uses random simulations to decide the best move.

Used in advanced game AI.

---

125. What are stochastic games?

Games where randomness is involved.

Example:

Ludo, Poker

---

126. Examples of stochastic games?

- Ludo
- Poker
- Dice games

---

127. What are partially observable games?

Games where complete information is not visible.

Example:

Poker

---

128. Examples of partially observable games?

- Poker
- Card games
- Strategy games with hidden moves

---

129. Limitations of game search algorithms?

- Very large search space
- High memory usage
- High time complexity
- Hidden information
- Randomness

Practical 4 – CSP / N-Queens / Graph Coloring

---

130. What is Constraint Satisfaction Problem (CSP)?

A problem where variables must satisfy given constraints.

---

131. Examples of CSP?

- N Queens
- Sudoku
- Graph Coloring
- Timetable Scheduling

---

132. What is Constraint Propagation?

Reducing possible values of variables using constraints.

This makes solving faster.

---

133. What is Backtracking Search?

Try a solution → if it fails → go back → try another option.

Very important question.

---

134. Why is backtracking used?

Because it helps solve problems where wrong choices must be undone and retried.

---

135. What is Branch and Bound?

It is an optimization technique that avoids useless branches using bounds.

It saves time.

---

136. Difference between Backtracking and Branch and Bound?

Backtracking:

- checks feasibility

Branch and Bound:

- focuses on optimization

Backtracking → valid solution

Branch and Bound → best solution

---

137. Difference between Branch and Bound and Dynamic Programming?

Branch and Bound:

- prunes unnecessary branches

Dynamic Programming:

- stores solved subproblems

They are different techniques.

Very common confusion question.

---

138. Is N-Queens a greedy problem?

No.

N-Queens mainly uses Backtracking.

Sometimes optimized using Branch and Bound.

It is not a greedy problem.

N-Queens

---

139. What is N-Queens problem?

Place N queens on an N × N chessboard so that no queen attacks another queen.

---

140. What are the attack conditions in N-Queens?

Queens should not be in:

- same row
- same column
- same diagonal

---

141. Why is backtracking used in N-Queens?

Because if one queen placement causes failure later, we go back and try another position.

---

142. Can Branch and Bound be used in N-Queens?

Yes.

It improves efficiency by skipping useless branches early.

---

143. How does Branch and Bound improve N-Queens?

It avoids checking obviously invalid positions and reduces unnecessary computation.

Graph Coloring

---

144. What is graph coloring?

Assigning colors to vertices so that no two adjacent vertices have the same color.

---

145. Why is graph coloring used?

To solve scheduling and allocation problems.

---

146. Applications of graph coloring?

- Exam scheduling
- Register allocation
- Map coloring
- Timetable generation

---

147. What is chromatic number?

Minimum number of colors needed to color a graph properly.

---

148. What is a Knowledge-Based Agent?

A Knowledge-Based Agent stores facts and rules in a knowledge base and uses them to make decisions.

It thinks using knowledge, not just fixed actions.

---

149. What is Wumpus World?

Wumpus World is a classic AI problem used to test logical reasoning.

An agent moves in a cave with:

- Wumpus (monster)
- Pits
- Gold

The goal is to find gold and come out safely.

Very common theory question.

---

150. Why is Wumpus World important in AI?

Because it helps explain:

- Knowledge representation
- Logical reasoning
- Decision making under uncertainty

---

151. What is Logic in AI?

Logic is a formal method of representing knowledge and drawing conclusions.

It helps machines reason like humans.

---

152. What is Propositional Logic?

It is logic using simple statements that are either True or False.

Example:

P = It is raining

---

153. Example of Propositional Logic?

P = It is raining

Q = Road is wet

$P \rightarrow Q$

If it rains, road becomes wet.

---

154. What is First Order Logic (FOL)?

FOL is a more powerful logic that uses:

- objects
- relations

- predicates
- quantifiers

It is more expressive than propositional logic.

---

155. Difference between Propositional Logic and FOL?

Propositional Logic:

- simple true/false statements

FOL:

- uses variables, predicates, quantifiers

FOL is more expressive.

Very common viva question.

---

156. What are predicates?

Predicates describe properties or relationships.

Example:

Human(x)

means x is human.

---

157. What are quantifiers?

Symbols used in FOL to express quantity.

Main types:

- Universal (∀)
- Existential (∃)

---

158. What is universal quantifier?

Symbol:

\forall

Means:

“For all”

Example:

All humans are mortal.

---

159. What is existential quantifier?

Symbol:

\exists

Means:

“There exists”

Example:

There exists a student who is intelligent.

---

160. What is theorem proving?

It is the process of proving whether a statement is true using logical rules.

---

161. What is model checking?

It checks whether a logical model satisfies given conditions.

Unit 5 – Reasoning

---

162. What is inference?

Inference means deriving new knowledge from existing facts.

---

163. What is unification?

Unification means matching variables and expressions in FOL.

It helps inference work properly.

---

164. What is Forward Chaining?

Forward Chaining is data-driven reasoning.

Start from facts → move toward conclusion.

Very common question.

---

165. What is Backward Chaining?

Backward Chaining is goal-driven reasoning.

Start from goal → check facts needed to prove it.

Very favorite viva question.

---

166. Difference between Forward and Backward Chaining?

Forward Chaining:

- starts from facts

Backward Chaining:

- starts from goal

Forward = data-driven

Backward = goal-driven

---

167. What is Resolution?

Resolution is a rule of inference used to prove statements logically.

It is used in theorem proving.

---

168. What is knowledge representation?

It is the method of storing knowledge in a form that AI systems can use for reasoning.

---

169. What is Ontological Engineering?

It is designing structured knowledge representation using categories, objects, and relationships.

---

170. What is default reasoning?

Assuming normal situations unless exception is specified.

Example:

Birds fly unless stated otherwise.

---

171. Example of default reasoning?

Normally:

Birds can fly

Exception:

Penguins cannot fly

This is default reasoning.

Practical 5 – Chatbot

---

172. What is chatbot?

A chatbot is a software program that simulates human conversation.

It interacts with users automatically.

---

173. Types of chatbot?

- Rule-based chatbot
- AI-based chatbot

---

174. Difference between rule-based and AI-based chatbot?

Rule-based:

- fixed responses
- follows predefined rules

AI-based:

- learns from data

- understands context better

---

175. Which chatbot did you implement?

Usually:

Rule-based chatbot

Say this confidently unless your project is different.

---

176. Why did you choose rule-based chatbot?

Because it is simple to implement, easy to explain, and suitable for academic practicals.

---

177. What is NLP?

NLP means Natural Language Processing.

It helps machines understand human language.

---

178. Applications of chatbot?

- Customer support
- Banking
- Healthcare
- E-commerce
- Education

---

179. Where are chatbots used?

- Websites
- Banking apps
- Customer service portals
- Hospitals
- Online shopping platforms

---

180. Can chatbot learn automatically?

Rule-based chatbot → No

AI-based chatbot → Yes

Depends on implementation.

Unit 6 – Planning

---

181. What is Automated Planning?

Automatically finding a sequence of actions to achieve a goal.

---

182. What is Classical Planning?

Planning in a known, deterministic environment where actions have predictable results.

---

183. What are heuristics in planning?

Rules or estimates used to make planning faster and smarter.

---

184. What is Hierarchical Planning?

Breaking one large problem into smaller sub-problems and solving step by step.

---

185. What is nondeterministic domain?

A domain where action results are uncertain and may vary.

---

186. Why is scheduling important in AI?

Because AI must manage:

- time
- resources
- tasks

efficiently.

---

187. What are AI components?

- Learning
- Reasoning
- Planning
- Perception
- NLP
- Robotics

---

188. What are AI architectures?

Frameworks used to design AI systems and define how AI components work together.

---

189. What are limits of AI?

- No emotions
- High cost
- Bias in decisions
- Data dependency
- Ethical concerns
- Lack of common sense

---

190. What is Ethics of AI?

Using AI responsibly with fairness, privacy, transparency, and safety.

Very modern favorite question.

---

191. Why is ethics important in AI?

Because wrong AI decisions can affect people's jobs, privacy, safety, and rights.

---

192. What is future scope of AI?

AI will grow in:

- healthcare
- robotics
- education
- finance
- self-driving vehicles
- cybersecurity
- automation

Huge future scope.

Final Universal Viva Questions

---

193. Explain your code without looking.

This tests real understanding.

You must explain:

- input
- logic
- algorithm
- output
- why each part is used

Very dangerous viva question.

---

194. Why did you choose this algorithm?

Because it is suitable for solving the given problem efficiently and matches the practical aim.

---

195. Can your code be optimized?

Yes.

Example:

Using better data structures, pruning unnecessary steps, reducing repeated calculations.

---

196. What are the limitations of your implementation?

- Works only for small inputs
- Limited error handling
- Basic implementation only
- Not optimized for real-world large systems

---

197. Real-life applications of your practical?

DFS/BFS → networks, maps

A* → GPS, games

N-Queens → scheduling

Chatbot → customer support

Greedy → routing, planning

---

198. Time complexity of your code?

Depends on algorithm:

DFS/BFS → $O(V^2)$

Dijkstra → $O(V^2)$

Kruskal → O(E log E)

Always explain according to your practical.

---

199. Space complexity of your code?

Depends on:

- arrays
- recursion stack
- queue
- open/closed lists

Usually based on number of vertices or states.

---

200. What if invalid input is given?

Program may fail.

We can improve it using:

- validation
- exception handling
- proper input checks

---

201. Can this problem be solved using another algorithm?

Yes.

Example:

DFS can use stack

A* can be replaced by Greedy or BFS depending on problem

N-Queens can use Branch and Bound

Always justify your answer.

---

202. Difference between theoretical and practical implementation?

Theory explains concept.

Practical implementation handles:

- input/output
- edge cases
- memory usage
- execution time
- real code problems
